	School of ICT **Sirindhorn International Institute of Technology**
CSS225 Operating System	**Homework 8: Virtual-Memory Management**

Part 1: Multiple-Choice Questions

1. Which one is INCORRECT about the virtual-memory management?
   A) It helps increasing the degree of multiprogramming.
   B) It does not allow that programs/processes can be larger than physical memory.
   C) It is a technique that allows the execution of processes that are not completely in memory.
   D) No incorrect answer (i.e., all answers are correct).

2. What is the demand paging?
   A) It is a shared memory architecture used in multiprocessing (multiple CPUs) systems.
   B) It is an electronic component that can operate a port, a bus, or a device.
   C) It is a strategy to load pages from a disk into the main memory only as they are needed.
   D) No correct answer.

3. What will happen if a process tries to access a page that was not brought into memory?
   A) A deadlock happens.
   B) Trashing happens.
   C) A page fault happens.
   D) No correct answer.

4. Which one is INCORRECT about copy-on-write?
   A) It is a technique allowing the parent and child processes initially to share the same pages.
   B) There is only one copy-on-write page per a process.
   C) A copy-on-write page means that, if either process writes to this shared page, a copy of the shared page is created.
   D) No incorrect answer (i.e., all answers are correct).

5. Assume that there exists a process with three user-level threads and that the mapping of user threads to kernel threads is many to one. If one user thread incurs a page fault while accessing its stack, how would the other user threads belonging to the same process also be affected by the page fault?
   A) They are not affected.
   B) They are in a deadlock.
   C) They have to wait for the faulting page to be brought into memory.
   D) No correct answer.

6. Which action could increase the page-fault rate?
   A) Increase the degree of multiprogramming
   B) Increase the size of the hard disk
   C) Increase the size of the main memory
   D) No correct answer

7. Assume that a page fault happens. If no frame is free, by using a _____, we find a frame that is not currently being used and free it.
   A) page-replacement algorithm
   B) victim frame
   C) demand paging
   D) No correct answer

8. Which one is CORRECT about the optimal page-replacement algorithm?
   A) The optimal page-replacement algorithm guarantees the lowest possible page fault rate for a fixed number of frames.
   B) The optimal page-replacement algorithm is difficult to implement.
   C) The optimal page-replacement algorithm requires future knowledge of the reference string.
   D) All answers are correct.

9. The _____ page-replacement algorithm uses the time when a page was brought into the memory.
   A) FIFO
   B) LRU
   C) Optimal
   D) No correct answer

10. Which page-replacement algorithm suffers from the Belady's anomaly?
   A) FIFO page-replacement algorithm
   B) Optimal page-replacement algorithm
   C) LRU page-replacement algorithm
   D) No correct answer

11. Which one is INCORRECT?
   A) Managing which page frames are stored at which locations can significantly affect performance in NUMA systems.
   B) In local replacement, a process cannot take a frame occupied by another process.
   C) The minimum number of frames is a factor considered in the allocation of frames.
   D) No incorrect answer (i.e., all answers are correct).

12. Consider that there are three processes: the process P0 with 15 pages, the process P1 with 30 pages, and the process P2 with 50 pages. If we would like to split 50 frames among these three processes, how many frames are given to each process by using the proportional allocation?
   A) P0 (16 frames); P1 (16 frames); P2 (16 frames)
   B) P0 (15 frames); P1 (30 frames); P2 (50 frames)
   C) P0 (8 frames); P1 (16 frames); P2 (26 frames)
   D) P0 (7 frames); P1 (15 frames); P2 (26 frames)

13. Which one is INCORRECT about thrashing?
   A) The system can detect thrashing by evaluating the level of CPU utilization as compared to the level of multiprogramming.
   B) Thrashing is caused by under-allocation of the minimum number of pages required by a process, forcing it to continuously page fault.
   C) Thrashing can be eliminated by increasing the level of multiprogramming.
   D) No incorrect answer (i.e., all answers are correct).

Part 2: Short-Answer Questions

1. (Exercise 9.3) Consider a system that uses pure demand paging:

   a) When a process first starts execution, how would you characterize the page fault rate?

Initially quite high as needed pages are not yet loaded into memory.

   b) Once the working set for a process is loaded into memory, how would you characterize the page fault rate?

It should be quite low as all necessary pages are loaded into memory.

   c) Assume a process changes its locality and the size of the new working set is too large to be stored into available free memory. Identify some options system designers could choose from to handle this situation?

(1) ignore it; (2) get more physical memory; (3) reclaim pages more aggressively due to the high page fault rate.

2. (Exercise 9.4) What is the copy-on-write feature, and under what circumstances is it beneficial? What hardware support is required to implement this feature?

When two processes are accessing the same set of program values (for instance, the code segment of the source binary), then it is useful to map the corresponding pages into the virtual address spaces of the two programs in a write-protected manner. When a write does indeed take place, then a copy must be made to allow the two programs to individually access the different copies without interfering with each other. The hardware support required to implement is simply the following: on each memory access, the page table needs to be consulted to check whether the page is write protected. If it is indeed write protected, a trap would occur and the operating system could resolve the issue.

3. (Exercise 9.7) When a page fault occurs, the process requesting the page must block while waiting for the page to be brought from disk into physical memory. Assume that there exists a process with five user-level threads and that the mapping of user threads to kernel threads is **one to one**. If one user thread incurs a page fault while accessing its stack, would the other user threads belonging to the same process also be affected by the page fault—that is, would they also have to wait for the faulting page to be brought into memory? Explain

Since the mapping of user threads to kernel threads is one to one, the other user threads do not have to wait for the faulting page.

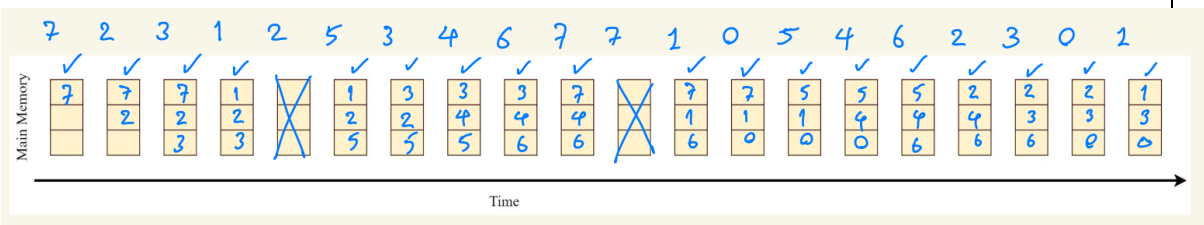
4. (Exercise 9.8) Consider the following page reference string:

7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 7, 1, 0, 5, 4, 6, 2, 3, 0 1.

Assuming demand paging with three frames, how many page faults would occur for the following replacement algorithms?

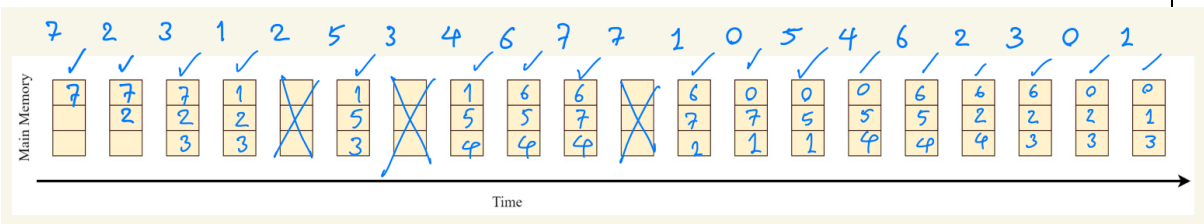
a) LRU replacement

Answer: Including the first three times when new coming-in pages are loaded into three empty frames, the number of page faults is 18 and, at the end, Pages 1, 3, 0 are in the memory (as shown below).



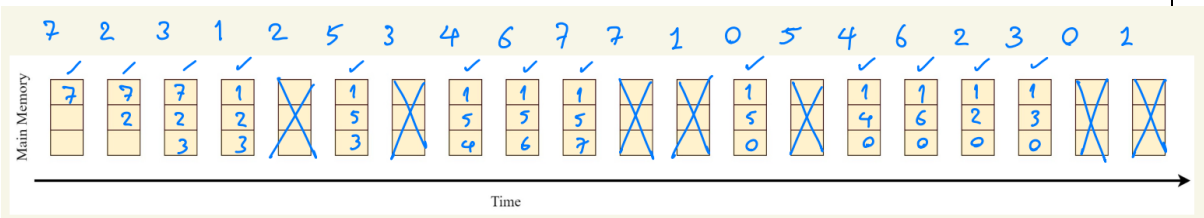
b) FIFO replacement

Including the first three times when new coming-in pages are loaded into three empty frames, the number of page faults is 17 and, at the end, Pages 0, 1, 3 are in the memory (as shown below).



c) Optimal replacement

Including the first three times when new coming-in pages are loaded into three empty frames, the number of page faults is 13 and, at the end, Pages 1, 3, 0 are in the memory (as shown below).



5. (Exercise 9.14) Consider a demand-paging system with the following time-measured utilizations:

CPU utilization 20%

Paging disk 97.7%

Other I/O devices 5%

For each of the following, indicate whether it will (or is likely to) improve CPU utilization. Explain your answers.

The system obviously is spending most of its time paging, indicating over-allocation of memory. If the level of multiprogramming is reduced resident processes would page fault less frequently and the CPU utilization would improve. Another way to improve performance would be to get more physical memory or a faster paging drum.

a) Install a faster CPU.

No.

b) Install a bigger paging disk.

No.

c) Increase the degree of multiprogramming.

No. This will increase the number of processes in the main memory.

d) Decrease the degree of multiprogramming.

Yes. This will lower the number of processes in the main memory and lower page faults.

e) Install more main memory.

Yes. Likely to improve CPU utilization as more pages can remain resident and not require paging to or from the disks.

f) Install a faster hard disk or multiple controllers with multiple hard disks.

Yes. Also an improvement, for as the disk bottleneck is removed by faster response and more throughput to the disks, the CPU will get more data more quickly.

g) Add pre-paging to the page-fetch algorithms.

Yes. Again, the CPU will get more data faster, so it will be more in use. This is only the case if the paging action is amenable to prefetching (i.e., some of the access is sequential).

h) Increase the page size.

No. More paging action could ensue because fewer pages can be kept in memory and more data is transferred per page fault. So this change is as likely to decrease CPU utilization.

6. (Exercise 9.19) What is the cause of thrashing? How does the system detect thrashing? Once it detects thrashing, what can the system do to eliminate this problem?

Thrashing is caused by under-allocation of the minimum number of pages required by a process, forcing it to continuously page fault. The system can detect thrashing by evaluating the level of CPU utilization as compared to the level of multiprogramming. It can be eliminated by reducing the level of multiprogramming.

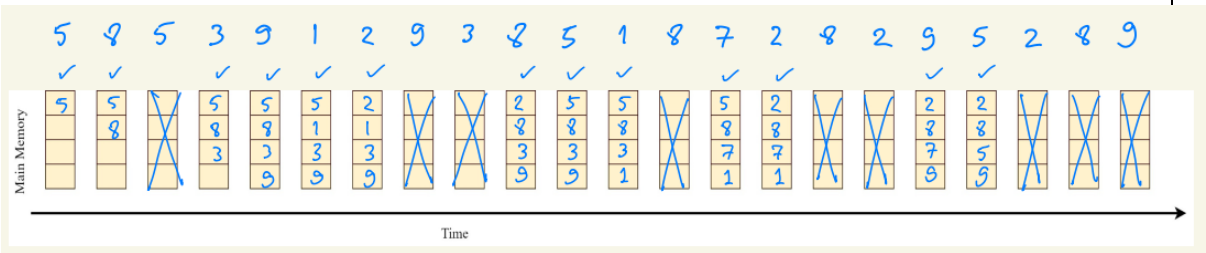
7. Assume that there are "four" empty frames (in the memory) allocated to this process. Consider the following reference string:

5, 8, 5, 3, 9, 1, 2, 9, 3, 8, 5, 1, 8, 7, 2, 8, 2, 9, 5, 2, 8, 9

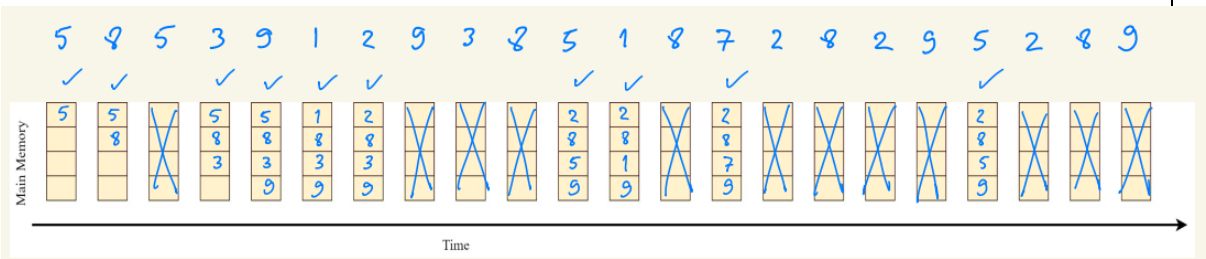
a) Find the number of page faults when using the LRU page-replacement algorithm. This number includes the first four times of page faults when new coming-in pages are loaded into four empty frames.

The number of page faults= _____13_____

Show your work to get the answer above; otherwise, points will be deducted.



b) Find the number of page faults when using the optimal page-replacement algorithm. This number includes the first four times of page faults when new coming-in pages are loaded into four empty frames.

The number of page faults= _____10_____

Show your work to get the answer above; otherwise, points will be deducted.